

Rumera Docker Infrastructure Documentation

This document explains the Docker setup for the Rumera project in a clear and structured way. It covers:

- Dockerfile (Go backend build process)
- docker-compose.yml (entire system architecture)
- networking
- caching strategy
- service communication
- volumes and persistence

1. System Overview

The system runs multiple services:

Service	Purpose
backend	Go API server
postgres	Main database
analytics_db	Time-series analytics database (TimescaleDB)
redis	caching / sessions
meilisearch	full-text search engine

All services communicate inside a Docker network:

rumera_network

2. Dockerfile (Go Backend)

Purpose

Build and run the Go backend efficiently with dependency caching.

Dockerfile

```
FROM golang:1.25-alpine AS builder

WORKDIR /app

RUN apk add --no-cache git

# 1. Copy dependency files first (CACHE LAYER)
COPY go.mod go.sum ./
RUN go mod download

# 2. Copy source code
COPY . .

# 3. Build binary
RUN go build -o server ./cmd/server/main.go

FROM alpine:latest

WORKDIR /app

RUN apk add --no-cache ca-certificates

COPY --from=builder /app/server .
COPY --from=builder /app/migrations ./migrations
COPY --from=builder /app/.env ./env

EXPOSE 8080

CMD [ "./server" ]
```

Dockerfile Explanation

Layer caching (VERY IMPORTANT)

```
COPY go.mod go.sum ./
RUN go mod download
```

This ensures:

- dependencies are cached

- rebuilds are fast
 - only code changes trigger rebuilds
-

Multi-stage build

- Stage 1: builds Go binary
- Stage 2: lightweight runtime image

Benefits:

- smaller image size
 - faster deployment
 - no build tools in final image
-

3. docker-compose.yml (Full System)

Purpose

Defines all services and how they connect.

Full Architecture

```
services:
  postgres:
    image: timescale/timescaledb:latest-pg17
    container_name: rumera_postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: rumera
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - rumera_network

  analytics_db:
    image: timescale/timescaledb:latest-pg17
    container_name: rumera_analytics
    restart: unless-stopped
```

```

environment:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgres
  POSTGRES_DB: rumera_analytics
ports:
  - "5433:5432"
volumes:
  - analytics_data:/var/lib/postgresql/data
networks:
  - rumera_network

redis:
  image: redis:8-alpine
  container_name: rumera_redis
  restart: unless-stopped
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  networks:
    - rumera_network

meilisearch:
  image: getmeili/meilisearch:v1.15
  container_name: rumera_meilisearch
  restart: unless-stopped
  environment:
    MEILI_MASTER_KEY: super_secret_key
    MEILI_ENV: development
  ports:
    - "7700:7700"
  volumes:
    - meili_data:/meili_data
  networks:
    - rumera_network

backend:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: rumera_backend
  restart: unless-stopped
  depends_on:
    - postgres
    - analytics_db
    - redis
    - meilisearch
  env_file:

```

```
- .env
ports:
  - "8080:8080"
volumes:
  - ./app
networks:
  - rumera_network

volumes:
  postgres_data:
  analytics_data:
  redis_data:
  meili_data:

networks:
  rumera_network:
    driver: bridge
```

4. Networking Model

All services communicate using Docker DNS:

Service	Internal Hostname
postgres	postgres
analytics	analytics_db
redis	redis
meilisearch	meilisearch

Example Go connection:

```
DB_HOST=postgres
```

NOT:

```
DB_HOST=localhost ❌
```

5. Volume Strategy

Volume	Purpose
postgres_data	main DB persistence
analytics_data	analytics persistence
redis_data	cache persistence
meili_data	search index persistence

Benefits:

- data survives container restart
- safe rebuilds
- production-like behavior

6. Meilisearch Role

Meilisearch runs as a separate service:

- Go uses it as a client
- It runs its own HTTP server
- Accessible at:

```
http://localhost:7700
```

Inside Docker:

```
http://meilisearch:7700
```

7. Build & Run Commands

Build everything

```
docker compose up --build
```

Run in background

```
docker compose up -d --build
```

Stop

```
docker compose down
```

Reset everything (DELETE DATA)

```
docker compose down -v
```

8. Development Workflow

Run migrations

```
docker compose exec backend make migrate-up
```

Analytics migrations

```
docker compose exec backend make analytics-up
```

Logs

```
docker compose logs -f backend
```

9. Key Design Principles

1. Dependency caching

Go modules are cached before copying source code.

2. Service isolation

Each service runs independently.

3. Internal DNS networking

Services communicate by name, not IP.

4. Persistent storage

Volumes prevent data loss.

5. Reproducibility

Entire system can be rebuilt with one command.

10. Summary

This Docker setup provides:

- scalable architecture
 - fast Go builds
 - persistent databases
 - analytics-ready TimescaleDB
 - search engine integration
 - production-like environment
-

This is not just a dev setup. It is a foundation for production-scale backend architecture.